

PRACTICAL REPORT

Blinking LED Control on the Microcontroller AT89C51

Content	Details
Microcontroller	AT89C51 (8051 family, 8-bit, 4 KB internal Flash)
Language	C (compiled with SDCC) — with an Assembly comparison version
Editor	Visual Studio Code
Compiler	SDCC 4.5.0 (Small Device C Compiler)
Simulation software	Proteus 9 Professional
Crystal oscillator	12 MHz → 1 machine cycle = 1 μs
Blink cycle	ON for 0.5 seconds — OFF for 0.5 seconds

1. Overview of the Toolchain

First point to understand: Visual Studio Code cannot compile or simulate the 8051 microcontroller. VS Code is only a text editor. The complete workflow requires three software tools working together:

Stage	Tool	Task
Source Code Editing	Visual Studio Code	Write main.c, provide syntax highlighting, and use a keyboard shortcut to invoke compilation
Compilation	SDCC	Convert C code into 8051 machine code and output a .hex file
Simulation	Proteus 9	Draw the schematic, load the .hex file, run the simulation, and observe the LED

Note: The .hex file (Intel HEX format) is the final output. The same file is used both for simulation in Proteus and for programming the physical chip.

2. Environment Setup

2.1. Installing the SDCC Compiler

Open Windows Terminal or PowerShell and run:

```
winget install SDCC.SDCC
```

After installation, SDCC is located at C:\Program Files\SDCC\bin. Verify it with:

```
sdcc --version
```

2.2. Project Directory Structure

```
8051_blink\  
├── main.c                C source code – main version  
├── main_timer0.c         Enhanced version using hardware Timer 0  
├── blink.asm             Assembly version for comparison  
├── build.bat             C → HEX build script  
├── build_asm.bat         ASM → HEX build script  
├── .vscode\  
│   ├── tasks.json        Defines build tasks for VS Code  
│   ├── settings.json     Disables CMake and configures this folder separately  
│   └── c_cpp_properties.json Library paths for IntelliSense  
├── build\  
│   ├── main.hex          ← FILE LOADED INTO PROTEUS  
│   ├── main.asm          Assembly code generated by SDCC  
│   ├── main.rel          Object file (addresses not yet fixed)  
│   ├── main.ihx          HEX file before packaging  
│   └── main.lst          Address – machine code – instruction mapping table
```

Important note: VS Code only reads the `.vscode` directory located at the workspace root (the folder opened with File → Open Folder). If `.vscode` is placed in a subfolder, VS Code will ignore it completely, and the build shortcut will fall back to the default task provided by another extension (usually CMake), resulting in an error.

3. Program Source Code

3.1. Contents of main.c

```
#include <8051.h>  
  
void delay_ms(unsigned int ms)  
{  
    unsigned int i, j;  
    for (i = 0; i < ms; i++)  
        for (j = 0; j < 120; j++)  
            ; /* empty loop, only used to consume time */  
}  
  
void main(void)  
{  
    P1_0 = 1; /* initial state: LED OFF */  
  
    while (1) /* microcontroller runs forever */  
    {  
        P1_0 = 0; /* kéo chân xuống 0V → LED SÁNG */  
        delay_ms(500);  
  
        P1_0 = 1; /* thả chân lên 5V → LED TẮT */  
        delay_ms(500);  
    }  
}
```

```
}
```

3.2. Operating Principle

a) Writing to a pin is a variable assignment. The AT89C51 has four ports, P0–P3, with 8 bits per port. In C, the statement `P1_0 = 0;` sets pin P1.0 (pin 1) to 0 V.

b) The LED must be connected in an active-LOW configuration. The 8051 port pin has asymmetric current sourcing and sinking capability:

Current Direction	Datasheet Specification	Value
Current sinking (output LOW)	IOL at VOL = 0.45 V	1.6 mA (maximum 10 mA)
Current sourcing (output HIGH)	IOH at VOH = 2.4 V	60 μ A (microamperes)

The difference is approximately 26 to 160 times. A typical LED requires 5–15 mA to glow clearly, so the LED must be connected from the positive supply to the microcontroller pin:

+5V — [R 330 Ω] —►|— P1.0 (chân 1)
LED

Command	Pin Voltage	LED State
<code>P1_0 = 0;</code>	0 V	ON — there is a voltage difference, so current flows through the LED
<code>P1_0 = 1;</code>	5 V	OFF — both ends of the LED are at the same voltage, so no current flows

c) A delay function is required. The microcontroller runs at 12 million cycles per second. If it is switched continuously without a delay, the human eye will only see the LED as continuously dim rather than blinking. The `delay_ms` function uses two nested empty loops to consume time.

d) An infinite loop is required. The microcontroller has no operating system to return to. If `main()` ends, the CPU will continue into unused memory and execute garbage data, resulting in unpredictable behavior. Therefore, the program must be enclosed in `while(1)`.

4. Compilation Process from C to HEX

The process consists of four sequential stages. The table below shows actual measurements from this project:

Step	Tool	Output File	Size	Content
—	(source code)	main.c	1,906 B	Human-readable text

1	SDCC compiler	main.asm	10,299 B	Assembly code, still text
2	sdas8051 assembler	main.rel	4,641 B	Binary, addresses not yet fixed
3	sdld linker	main.ihx	654 B	Actual addresses assigned
4	packihx packaging tool	main.hex	732 B	File loaded into the chip

Step 1 — Compiler: C to Assembly

SDCC reads the C code and generates the corresponding machine instructions while retaining the original C lines as comments for easier comparison:

```
; main.c:44: P1_0 = 0;
clr    _P1_0                ; một dòng C → một lệnh máy

; main.c:45: delay_ms(500);
mov     dptr,#0x01f4         ; 0x01F4 = 500, load the parameter into DPTR
lcall   _delay_ms            ; then call the function
```

The last two lines clearly show the parameter-passing mechanism: the value 500 does not enter the function automatically. The compiler must first load it into the DPTR pointer register, and the called function reads it from there.

Step 2 — Assembler: Text to Numbers

Each Assembly instruction is converted into machine code according to the fixed 8051 opcode table:

ADDRESS	MACHINE CODE	ORIGINAL INSTRUCTION
000030	D2 90	setb P1_0
000032	C2 90	clr P1_0
000034	12 00 3E	lcall delay_500ms
00003C	80 F4	sjmp main_loop

Here, D2 is the opcode for SETB, and 90 is the bit address of P1.0. At this stage, the addresses are still relative because the final placement of the code is not yet known.

Step 3 — Linker: Assigning Absolute Addresses

The sdld program combines the .rel files (user code plus the SDCC startup library), determines the location of each section in memory, and then fills in the actual values at the marked locations.

Step 4 — Packaging into Intel HEX Format

A HEX file is not raw machine code. It is machine code rewritten as hexadecimal text, together with addresses and a checksum, so that it can be transmitted through a serial interface with error checking.

Analysis of the first line of blink.hex:

```
:03 0000 00 020030 CB
```

Component	Value	Meaning
:	—	Every line begins with a colon
03	3	This line contains 3 data bytes
0000	0x0000	Load at address 0x0000
00	data	Record type: 00 = data, 01 = end of file
02 00 30	—	These are the 3 machine-code bytes
CB	—	Checksum

The three bytes 02 00 30 decode as follows: 02 is the LJMP opcode, followed by the destination address 0030. This means: "at address 0, jump to address 0x0030."

Checksum calculation — add all bytes and take the two's complement:

```
03 + 00 + 00 + 00 + 02 + 00 + 30 = 0x35
0x100 - 0x35 = 0xCB      ✓ matches the final byte
```

The programmer recalculates the checksum; if even one bit differs, the file is considered corrupted and rejected. The final line of every HEX file is always :00000001FF — record type 01 indicates the end of the file.

5. Compiling in VS Code

Step	Action
1	Open VS Code, select File → Open Folder, and navigate to the project directory
2	Press Ctrl + Shift + B to run the default build task
3	Observe the terminal and wait for the message THANH CONG! (SUCCESS!)
4	Check the output file at build\main.hex

To compile another version (main_timer0.c or blink.asm), press Ctrl + Shift + P, type Tasks: Run Task, and select the correct task name from the list.

Difference between the two commands: Ctrl+Shift+B directly runs the task marked as default without prompting. Tasks: Run Task lists all tasks, including tasks provided by other extensions.

6. Simulation in Proteus

6.1. Creating a New Project

Step	Action
1	Launch Proteus 9 Professional
2	Select File → New Project, enter a name, and choose a save location
3	At the template selection step, check Create a schematic from the selected template → select DEFAULT
4	For the following steps, select Do not create a PCB layout and No Firmware Project, then click Finish

6.2. Selecting Components

Press P (or click the Pick Devices button with the P icon) to open the library, then search for and select the following components:

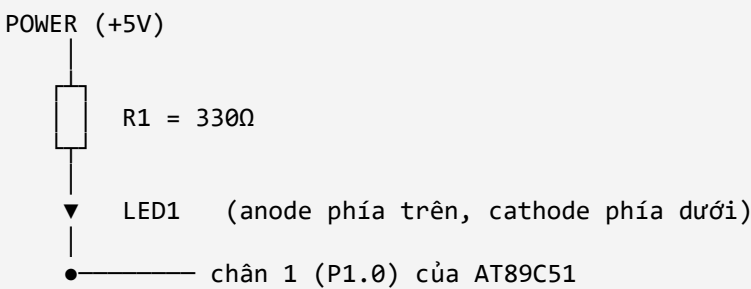
Search Keyword	Component	Notes
AT89C51	Microcontroller	Select the correct 40-pin version
LED-RED	Red LED	Other colors may be selected
RES	Resistor	Its value will be set to 330 Ω later

The power and ground terminals are not in the component library. Click the Terminals Mode icon on the left vertical toolbar, then select:

- **POWER** — positive terminal +5V
- **GROUND** — ground 0V

6.3. Drawing the Schematic

Drag the components into the workspace and wire them according to the schematic:



Wiring method: move the cursor to a component pin until a red square appears, left-click, drag to the pin to be connected, and click again.

Set the resistor value: double-click R1, enter 330 in the Resistance field, and click OK.

Regarding the oscillator and reset circuits: in Proteus, the crystal, capacitors, and reset circuit do not need to be drawn — the software simulates them automatically. They only need to be installed when building the physical circuit (see Appendix A).

6.4. Loading the HEX File and Setting the Frequency

Step	Action
1	Double-click the AT89C51 to open the Edit Component dialog
2	In the Program File field, click the folder icon on the right
3	Navigate to build\main.hex in the project directory and click Open
4	In the Clock Frequency field, enter 12 MHz
5	Click OK to close the dialog

6.5. Running the Simulation

Click the Play button (triangle ►) in the lower-left corner of the window. Expected result: the LED turns ON for 0.5 seconds and OFF for 0.5 seconds, repeating continuously until the Stop button (square ■) is clicked.

On the schematic, the microcontroller pins display small squares that change color according to the logic level: red indicates HIGH (5 V), and blue indicates LOW (0 V). Pin P1.0 changes color in sync with the LED.

7. Extended Experiments

Source Code Change	Observed Result
Change delay_ms(500) to delay_ms(100)	LED blinks 5 times faster
Remove the delay_ms call completely	LED appears continuously dim — because the state changes too quickly for the human eye
Change P1_0 to P2_3	LED moves to pin P2.3; the wiring in Proteus must be changed
Use P1 = 0x00; and P1 = 0xFF;	Controls all 8 pins of port P1 simultaneously

8. Conclusion

The practical exercise achieved the following objectives:

- Established the complete VS Code — SDCC — Proteus toolchain for 8051 programming
- Understood the four stages of converting C source code into a programming file: compilation, assembly, linking, and packaging
- Decoded the structure of an Intel HEX file and understood how to verify its integrity using a checksum
- Understood the active-LOW LED connection principle based on the current characteristics of the 8051 port
- Successfully simulated the LED blinking circuit in Proteus

Appendix A — Required Pins for a Physical Circuit

Proteus simulates these automatically, so they do not need to be drawn, but the physical circuit will not work if a required pin is missing:

Pin	Number	Connection
VCC	40	Connect to +5 V
GND	20	Connect to 0 V
EA	31	Connect to +5 V — the most commonly forgotten pin. See the explanation below
XTAL1 / XTAL2	19 / 18	Connect both ends of the 12 MHz crystal; add a 33 pF capacitor from each end to ground
RST	9	Connect a 10 μ F capacitor to VCC and a 10 k Ω resistor to ground

Why does only the EA pin need to be connected, while ALE and PSEN do not?

These three pins all serve external memory expansion. The difference lies in the signal direction:

Pin	Number	Direction	Function
EA	31	Input	Determines where instruction code is fetched from. Connect to +5 V = use internal Flash; connect to ground = read entirely from external ROM
ALE	30	Output	Address latch pulse for an external latch IC when using expanded memory
PSEN	29	Output	Instruction read pulse from external program ROM

EA is an input, so the chip requires a defined level and it must be connected to a fixed level. If left floating, the voltage on the pin is random; the chip may misread it and attempt to fetch instructions from nonexistent external memory, resulting in unpredictable operation.

ALE and PSEN are outputs — the chip itself generates these signals and does not wait for anything from outside. When external memory is not used, leaving them unconnected is completely normal.

Appendix B — Common Troubleshooting

Symptom	Cause and Solution
When Ctrl+Shift+B is pressed, VS Code runs CMake, builds, and then reports an error	tasks.json cannot be found at the workspace root, so VS Code uses a task provided by an extension. Check whether .vscode is located in the directory currently opened.
Error: The filename, directory name, or volume label syntax is incorrect	The path in tasks.json uses single backslashes. In JSON, \ is an escape character, so \b is interpreted as backspace. Use forward slashes / instead.
Error: 'build.bat' is not recognized even though the correct directory is set	The system has the environment variable NoDefaultCurrentDirectoryInExePath=1, so cmd does not automatically search the current directory. The full path must be provided.
Proteus runs but the LED does not light	Check whether the correct .hex file was loaded, whether Clock Frequency is set to 12 MHz, and whether the LED is connected in the correct active-LOW orientation.
LED appears continuously dim instead of blinking	The delay function is missing or incorrectly placed. The delay must be between the two state changes, not after both.
The physical circuit does not work even though the Proteus simulation works	Most likely, the EA pin (31) was not connected to +5 V, or the crystal and reset circuit are missing.